

10주차 세션 요약본

Efficient Memory Management for Large Language Model Serving with PagedAttention

1. Introduction

분야	LLM Serving 시스템에서의 효율적인 메모리 관리
목표	KV cache 메모리를 효율적으로 관리하여 LLM serving throughput 향상
응용	GPT-PaLM 기반 프로그래밍 어시스턴트, 챗봇 등 클라우드 호스팅 서비스
배경 수치	LLM 요청 처리 비용은 기존 키워드 검색 대비 10배 이상 비쌈. NVIDIA A100 40GB에서 Parameters 65%, KV Cache >30% 차지

LLM 동작 방식

- Autoregressive Transformer 기반 → 토큰을 하나씩 순차 생성
- 매 step마다 이전 모든 토큰의 key, value vector(KV cache)가 필요
- Sequential 생성 구조로 workload가 memory-bound → GPU 연산 활용 낮음, throughput 제한

기존 연구의 한계 (FasterTransformer, Orca)

근본 원인: KV cache를 연속된 메모리(contiguous memory)에 저장 → 두 가지 비효율 발생

- Internal & External Fragmentation
 - 요청의 최대 길이(예: 2048 tokens) 기준으로 미리 메모리 할당(pre-allocation)
 - 실제 출력 길이는 최대값보다 훨씬 짧은 경우가 많음 → internal fragmentation
 - 요청마다 pre-allocated 크기가 달라 메모리 할당자에서 external fragmentation 발생
 - 예약된 공간도 요청 완료 시까지 다른 요청이 사용 불가
- Memory Sharing 불가
 - Parallel sampling, beam search 등은 여러 sequence가 KV cache 일부를 공유 가능하나, 각 sequence의 KV cache가 별도 contiguous space에 저장되어

공유 불가

- 기존 시스템에서 실제 token state에 사용되는 KV cache 비율은 20.4% ~ 38.2% 수준에 불과

Contributions

- OS의 virtual memory & paging에서 영감을 받은 PagedAttention + 이를 기반으로 한 LLM serving engine vLLM 제안
- near-zero waste in KV cache memory + flexible sharing of KV cache within and across requests
 1. LLM serving에서 메모리 할당 문제를 식별하고 serving 성능에 미치는 영향 정량화
 2. PagedAttention 제안
 3. vLLM 설계 및 구현
 4. 다양한 시나리오 평가 (Basic Sampling, Parallel Sampling, Beam Search, Shared Prefix 등)

2. Background

KV Cache란?

- LLM이 토큰을 생성할 때 각 step에서 이전 모든 토큰의 key, value vector를 재계산하지 않고 저장해두는 것

KV Cache의 두 가지 생성 단계

1. Prompt phase: 전체 입력 prompt를 한 번에 처리 → matrix-matrix multiplication → GPU parallelism 효율적 활용
2. Autoregressive generation phase: 매 iteration마다 새 token 1개씩 생성 → matrix-vector multiplication → GPU 연산 활용 낮음, memory-bound, 단일 요청 latency의 대부분을 차지

Batching

- 여러 요청을 묶어 처리하면 model weights 이동 overhead가 분산되어 throughput 향상 가능
- Iteration-level scheduling: 매 iteration마다 완료된 요청 제거 + 새 요청 추가 → queueing delay 감소, padding 제거 → fine-grained batching으로 throughput 향상

3. Memory Challenges in LLM Serving

1. Large KV cache: 요청 수가 늘수록 KV cache 크기 급격히 증가 → 메모리가 점점 더 큰 bottleneck
2. Complex decoding algorithms: 기존 시스템(parallel sampling, beam search)은 sharing 기회를 전혀 활용하지 못함
3. Scheduling for unknown input & output lengths: 출력 길이를 사전에 알 수 없음 → 메모리 부족 시 일부 요청의 KV cache를 삭제하거나 swap out하는 scheduling 결정 필요

4. Method

PageAttention

- OS의 virtual memory & paging을 KV cache 관리에 적용
 - blocks = pages, tokens = bytes, requests = processes
 - 각 sequence의 KV cache를 고정 크기의 KV block으로 분할
 - 각 block은 고정된 수(block size B)의 토큰에 대한 key, value vector 저장
 - KV block들은 non-contiguous physical memory에 저장 가능
- 기존 attention을 block-wise computation으로 변환, PagedAttention kernel이 block table을 참조하여 각 KV block을 개별적으로 fetch하여 연산

효과

- internal fragmentation → 작은 block 사용 + on-demand 할당으로 완화
- external fragmentation → 모든 block이 동일한 크기이므로 제거
- memory sharing → block 단위로 가능

KV Cache Manager & Block

KV Cache Manager 구조

- logical KV blocks → physical KV blocks 매핑 (OS virtual memory와 동일 개념)
- Block table: 각 요청의 logical block ↔ physical block 매핑 + filled position 수 기록
- logical-physical 분리 → 필요할 때만 physical block 동적 할당 → 메모리 낭비 대부분 제거

- 요청당 메모리 낭비는 마지막 block 하나 이내로 제한, 요청 완료 시 즉시 해제

다양한 디코딩 시나리오 적용

• Parallel Sampling

- 하나의 input prompt에서 여러 output 생성하는 경우
- prompt KV cache는 모든 sample이 동일 → 하나의 physical block에 저장 후 공유.
- 각 block에 reference count 도입.
- 마지막 logical block만 copy-on-write, 나머지 prompt block은 완전 공유 → long prompt일수록 메모리 절약 효과 큼

• Beam Search

- 상위 k개의 output sequence를 유지하며 디코딩
- prompt block뿐 아니라 중간 block도 candidate 간 공유 가능.
- sharing pattern이 decoding 진행에 따라 동적으로 변화 → OS의 compound fork process tree와 유사

• Shared Prefix

- 사용자가 task description + 예시 입출력(system prompt)을 긴 prefix로 제공하는 경우
- LLM service provider가 predefined shared prefix의 physical block을 미리 예약.
- 해당 prefix를 포함한 요청은 logical block을 미리 캐싱된 physical block에 직접 매핑 → prefix 계산 중복 제거 (OS의 shared library 개념과 동일)

+Mixed Decoding Methods

Scheduling & Preemption & Distributed Execution

Scheduling 정책

- FCFS (First-Come-First-Serve) → 공정성 보장, starvation 방지
- Preemption 발생 시: 가장 늦게 도착한 요청을 먼저 preempt

Preemption 발생 조건 및 정책

- GPU의 physical block이 부족해지면 일부 요청을 preempt

- All-or-nothing eviction policy: 한 sequence의 block은 전부 evict하거나 전부 유지
- Beam search처럼 memory sharing이 있는 sequence group은 gang-scheduled → 항상 함께 preempt/reschedule
- 복구 방법 2가지: Swapping (CPU로 이동) / Recomputation (재계산)

Distributed Execution

- 대형 모델은 single GPU 용량 초과 → Megatron-LM 방식의 tensor model parallelism 지원 (SPMD)
- attention operator는 attention head 차원으로 분할, GPU 간 all-reduce로 중간 결과 동기화
- 단일 KV cache manager를 centralized scheduler 내에 유지 → 각 GPU worker가 동일한 logical-physical block mapping 공유

5. Experiment

실험 설정

- 모델: OPT-13B / OPT-66B / OPT-175B / LLaMA-13B
- 하드웨어: Google Cloud A2 instances, NVIDIA A100 GPU (13B: 1개, 66B: 4개, 175B: 8개)
- 데이터셋: ShareGPT (평균 input 161토큰 / output 338토큰), Alpaca (평균 input 19토큰 / output 58토큰)
- Baselines: FasterTransformer, Orca (Max/Pow2/Oracle)
- 핵심 지표: Normalized latency (s/token) — 높은 request rate에서도 낮은 normalized latency 유지

결과 요약

- FasterTransformer·Orca 대비 2~4× throughput 향상 (모델 정확도 변화 없음)
- 긴 sequence, 큰 모델, 복잡한 decoding algorithm일수록 개선 효과 더 크게 나타남
- vLLM: ShareGPT에서 최대 42개, Alpaca에서 최대 132.44개 요청을 동시에 배치 처리

6. Ablation Studies

Kernel Microbenchmark

- 전용 GPU kernel 3종 개발: Fused reshape and block write / Fusing block read and attention / Fused block copy
- FasterTransformer 대비 attention kernel latency 20~26% 증가 (overhead는 attention operator에만 국한)
- end-to-end 성능에서는 PagedAttention 덕분에 vLLM이 FasterTransformer를 크게 앞섬

Impact of Block Size

- block size 너무 작으면 → GPU parallelism 활용 부족
- block size 너무 크면 → internal fragmentation 증가 + sharing 확률 감소
- block size 16 → 대부분의 workload에서 GPU 효율적 활용 + internal fragmentation 최소화의 균형점 → vLLM 기본값으로 설정

Comparing Recomputation and Swapping

- block size 작을 때: CPU-GPU 간 소규모 데이터 전송 증가 → PCIe bandwidth 효율 저하 → recomputation 유리
- block size 클 때: swapping 유리
- block size 16~64 중간 범위 → 두 방법의 end-to-end 성능 유사
- recomputation overhead는 swapping latency의 최대 20%를 넘지 않음

7. Conclusion

핵심 contribution 요약:

1. LLM serving에서 KV cache 메모리 낭비 문제를 식별하고 정량화
2. OS의 virtual memory & paging에서 영감을 받은 PagedAttention 제안 → non contiguous paged memory에서 KV cache 저장
3. vLLM 구현 → near-zero waste in KV cache memory + flexible KV cache sharing (within/across requests)
4. FasterTransformer, Orca 대비 2-4× throughput 향상 (모델 정확도 변화 없음), 긴 sequence, 큰 모델, 복잡한 decoding algorithm일수록 개선 효과 더 크게 나타남

Adam: A Method for Stochastic Optimization

1. Introduction

연구 배경

- 딥러닝의 목적 함수(loss function)는 본질적으로 stochastic(확률적)
- 미니배치마다 다른 sub-function 합으로 구성 → 매 스텝 그래디언트가 달라짐
- Dropout 같은 정규화는 추가 noise 주입 → 더 강건한 옵티마이저 필요
- 1차 미분 기반(First-order) 방법: 빠르지만 방향 정보가 제한적
- 2차 미분(Hessian) 활용 시 역행렬 계산이 $O(d^2 \sim d^3)$ → 수백만 파라미터에 비현실적

→ 결론: 효율적이고 noise에 강건한 1차 stochastic 옵티마이저가 필요

옵티마이저 계보

계열	알고리즘	특징
방향성	Momentum (1964), NAG (1983)	관성을 활용해 진동 줄이고 가속. 과거 그래디언트를 누적해 진행
스텝 사이즈	AdaGrad (2011), RMSProp (2012), AdaDelta (2012)	파라미터별 학습률을 적응적으로 조절. AdaGrad는 누적 제곱 그래디언트, RMSProp은 지수 이동평균으로 개선
방향성+스텝	Adam (2014), Nadam (2016)	두 갈래의 장점 결합. Adam은 그 중 Momentum + RMSProp 통합

논문의 Contribution

1. 구현이 간단하고 연산이 효율적이며 메모리 요구량이 거의 없음
2. 그래디언트의 대각 재스케일링(diagonal rescaling)에 영향받지 않음 → 파라미터별 학습률 자동 조정
3. 데이터 또는 파라미터가 큰 대규모 문제에 적합
4. 잡음(noisy)·희소(sparse) 그래디언트에 강건
5. 하이퍼파라미터(β_1, β_2)가 직관적이고 튜닝이 거의 불필요
6. 실험에서 SGD·AdaGrad·RMSProp 대비 일관된 우수 성능

2. Related Work

AdaGrad

- 가중치 업데이트 횟수에 따라 학습률을 자동 조절하는 적응형 옵티마이저
 - 적게 업데이트된 가중치는 h 가 작음 $\rightarrow$ 학습률 커짐 / 많이 업데이트된 가중치는 h 가 누적증가 $\rightarrow$ 학습률 작아짐
 - 장점: Sparse gradient(희소한 특징)에 매우 적합
 - 한계: 과거 기울기 제곱을 무한 누적 $\rightarrow h$ 가 단조 증가 $\rightarrow$ 학습률이 0에 수렴 $\rightarrow$ 장기 학습 시 업데이트 정지

RMSProp

- AdaGrad의 학습률 소실 문제를 지수 이동 평균(EMA)으로 해결
 - Decaying Rate (ρ)를 사용하여 먼 과거의 기울기는 지수적으로 잊고 최근의 기울기를 크게 반영
 - $\rho \approx 0.9 \rightarrow$ 직전 ~ 10 스텝의 그래디언트 분산을 추적 $\rightarrow$ AdaGrad의 학습률 0 수렴 문제 해소
 - Adam의 2차 moment 추정에 그대로 계승됨

Adam의 차별성

- Adam = Adaptive Moment estimation.
- 1차 moment(그래디언트 평균 $\rightarrow$ 방향 정보)와 2차 moment(그래디언트 제곱 평균 $\rightarrow$ 크기 정보)를 동시에 추정하여 파라미터별 학습률 결정
 - AdaGrad의 sparse gradient 처리 능력
 - RMSProp의 on-line·non-stationary 적응력
 - Momentum의 방향 누적 + 편향 보정 결합

3. Algorithm

Adam 알고리즘 (기본값: $\alpha=0.001$, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$)

1. 그래디언트 계산

$$g_t = \nabla \theta f_t(\theta_{t-1})$$

- 미니배치 손실의 미분

2. 모멘트 갱신

$$m_t = \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot g_t \quad v_t = \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot g_t^2$$

- 방향(1차 EMA) + 크기(2차 EMA) 업데이트

3. 편향 보정

$$\hat{m}_t = m_t / (1 - \beta_1^t) \quad \hat{v}_t = v_t / (1 - \beta_2^t)$$

- 초기 0-편향 제거. $t=1$, $\beta_1=0.9$ 에서 보정 없으면 추정치가 실제값의 10%에 불과

4. 파라미터 업데이트

$$\theta_t = \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

- 상한 $|\Delta\theta_t| \leq \alpha$ — 학습률이 무한대로 발산하지 않음

SNR (Signal-to-Noise Ratio)

- $\hat{m}_t / \sqrt{\hat{v}_t} \approx$ 신호 대 잡음비(SNR)
 - SNR $\uparrow$ (그래디언트 일관) $\rightarrow$ 큰 스텝, 빠르게 전진
 - SNR $\downarrow$ (잡음 큼 / 최적값 근처) $\rightarrow$ 작은 스텝, 자동 어닐링 $\rightarrow$ LR 스케줄 불필요

Step Size 상한

- Case 1 (Sparsity): $(1 - \beta_1) > \sqrt{(1 - \beta_2)} \rightarrow |\Delta_t| \leq \alpha \cdot (1 - \beta_1) / \sqrt{(1 - \beta_2)}$ — 희소 그래디언트 시 큰 보폭 허용
- Case 2 (일반): $(1 - \beta_1) \leq \sqrt{(1 - \beta_2)} \rightarrow |\Delta_t| \leq \alpha$ — 기본값 9/0.999가 이 영역. α 는 trust region 역할

효율적 구현 형태

- 편향 보정을 학습률에 흡수: $\alpha_t = \alpha \cdot \sqrt{(1 - \beta_2^t)} / (1 - \beta_1^t) \rightarrow \theta_t = \theta_{t-1} - \alpha_t \cdot m_t / (\sqrt{v_t} + \epsilon)$
 - $\hat{m}$, $\hat{v}$ 를 따로 저장할 필요 없음 $\rightarrow$ 메모리 절감
 - 나눗셈 1회로 단축 $\rightarrow$ 대규모 모델에서 wall-clock 속도 향상
- PyTorch / TensorFlow 등 주요 프레임워크의 Adam 구현이 모두 이 형태를 채택

4. Experiments

MNIST — Multi-class Logistic Regression

- L2-regularized multi-class logistic regression, mini-batch 128, step size $1/\sqrt{t}$ decay
- Adam은 SGD와 유사한 수렴 속도, AdaGrad보다 빠른 수렴 $\rightarrow$ dense feature 환경에서도 안정적으로 동작

IMDB — Sparse Feature 환경

- BoW(Bag of Words) Feature Vector로 변환 $\rightarrow$ 10,000 차원, 50% drop-out 적용

- sparse 환경에서 Adam, RMSProp, AdaGrad 모두 우수한 성능. Adam은 SGD보다 빠른 수렴 속도

MLP — Multi-Layer Neural Networks

- 2 fully-connected hidden layers × 1000 units, ReLU, mini-batch 128, Cross-Entropy + L2 + dropout
- Adam이 가장 낮은 training cost로 빠르게 수렴, 2차 근사 기반 SFO 대비 iterations·wall-time 모두 우위

CNN — CIFAR-10 (단기 3 epochs / 장기 45 epochs)

- C64-C64-C128-FC1000, ReLU + dropout 적용/미적용 모두 비교
- 초기 3 epoch: Adam이 가장 빠르게 cost 감소 (AdaGrad는 정체)
- 장기 45 epochs: Adam, SGDNesterov는 10^{-3} 부근에서 급격한 성능 향상 후 안정 수렴. AdaGrad는 누적 제곱 그래디언트로 인해 학습률이 지나치게 작아져 수렴 정체

Bias Correction 효과 검증

- β_2 가 1에 가까울수록(0.9999) 보정 미적용 시 초기 발산 — bias correction 효과가 결정적
- 10 epoch 단기에서 차이가 명확, 100 epoch 후에는 모든 설정에서 비슷하게 수렴
- 결론: bias correction은 학습 초반 안정성에 필수적인 요소

5. Ablation Studies

AdaMax

- Adam의 v_t (L^2 -norm 기반 추정)를 L^p -norm으로 일반화.
- $p \rightarrow \infty$ 극한(L^∞ -norm = max norm)에서 단순하고 안정적인 형태로 수렴
 - $u_t = \max(\beta_2 \cdot u_{t-1}, |g_t|)$ — 과거 추적값과 현재 그래디언트 중 큰 쪽 선택
 - 파라미터 갱신: $\theta_t = \theta_{t-1} - (\alpha / (1 - \beta_1^t)) \cdot \hat{m}_t / u_t$
- 장점: u_t 는 0이 될 수 없음 $\rightarrow \epsilon$ (분모 안정화 항) 불필요, 식이 단순
- v_t 에 대한 bias correction 불필요 (max 연산이 0으로 초기화 편향되지 않음)
 - 권장 기본값: $\alpha=0.002$, $\beta_1=0.9$, $\beta_2=0.999$

Temporal Averaging

- SGD 마지막 iterate는 노이즈에 흔들림.

- 이론적으로 Polyak-Ruppert averaging이 더 좋은 일반화 성능을 보장하나, 모든 θ_t 를 저장·평균해야 해 비용이 큼
- 지수가중 이동평균을 제안 — $\bar{\theta}_t = \beta_2 \cdot \bar{\theta}_{t-1} + (1-\beta_2) \cdot \theta_t$, bias correction: $\hat{\theta}_t = \bar{\theta}_t / (1-\beta_2^t)$
- 추가 메모리 1배만으로 수렴 안정성과 일반화 성능 향상 — Adam·AdaMax에 그대로 결합 가능

6. Conclusion

Adam의 강점

1. 효율성: $O(d)$ 메모리·계산, 대규모 모델에 최적화
2. 스케일 불변성: 그래디언트 대각 재스케일링에 견고
3. 직관적 HP: $\alpha=0.001$, $\beta_1=0.9$, $\beta_2=0.999$ 기본값으로 대부분 작동
 - 핵심: 편향 보정 = 작지만 결정적 기여 — 기존 방법(Momentum + RMSProp)의 장점을 완전 통합
 - BERT·GPT·ResNet·GAN 등 최신 딥러닝 모델의 사실상 표준 옵티마이저(De facto Standard)
 - 딥러닝 역사상 가장 많이 인용된 논문 중 하나 (120,000+ Citations)